ECE30030/ITP30010 Database Systems

SQL DDL

Reading: Chapter 3

Charmgil Hong

charmgil@handong.edu

Spring, 2026

Handong Global University

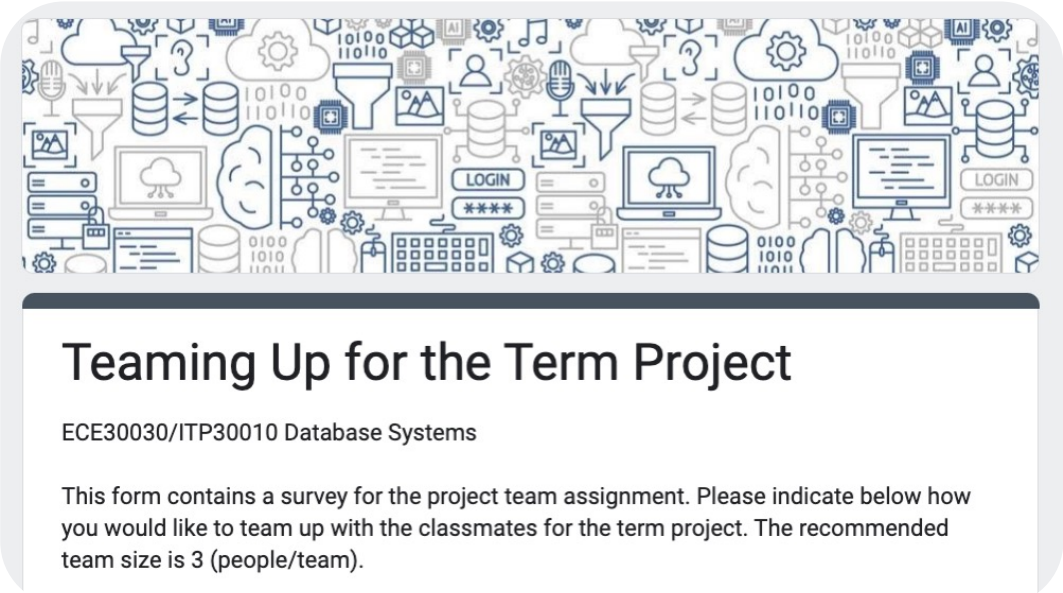
Announcements

- Study material #3 is posted

- Quiz #2 is on April 9, 2026 (this coming Thursday)
  - Regular class hours
  - Closed-book
  - Topics (~ April 6 coverage)
    - SQL DML
    - SQL DDL

Announcements

- Make teams for the term project
  - https://forms.gle/mYJW7xU86Sv5ZD7H7 Reponse due: Thursday, April 16
  - Problem & data release: Week #8 (tentative)



Announcements

- TA office hours are available on a regular basis
  - Monday 14:00-15:00 -- Eunchan Jung (NTH217; 22100685@handong.ac.kr)
  - Tuesday 19:00-20:00 -- Hyunmo Kang (NTH217; 22100026@handong.ac.kr)
  - Thursday 14:00-15:00 -- Yunseong Choi (NTH221; yunseong.choi@handong.ac.kr)
  - Friday 14:00-15:00 -- Yujin Kim (NTH221; yujin.kim@handong.ac.kr)

  - If you find any part of the material challenging or would like additional help, please feel encouraged to visit

Aggregate Functions Examples

- Find the average salary of instructors in the Computer Science department
  - **SELECT AVG**(*salary*)
    FROM *instructor*
    WHERE *dept_name*= 'Comp. Sci.';

`AVG(salary)`
77333.333333

- Find the total number of instructors who teach a course in the Spring 2018 semester
  - **SELECT COUNT**(**DISTINCT** *ID*)
    FROM *teaches*
    WHERE *semester* = 'Spring' **AND** *year* = 2018;

`COUNT(DISTINCT ID)`
6

- Find the number of tuples in the *teaches* relation
  - **SELECT COUNT** (*)
    FROM *teaches*;

`COUNT(*)`
15

Aggregate Functions – Having Clause

- Note: predicates in the **HAVING** clause are applied after the formation of groups whereas predicates in the **WHERE** clause are applied before forming groups

SELECT *dept_name*, **AVG**(*salary*) **AS** *avg_salary*
FROM *instructor*
GROUP BY *dept_name*
HAVING AVG(*salary*) > 65000;

dept_name	avg_salary
Biology	72000.000000
Comp. Sci.	77333.333333
Elec. Eng.	80000.000000
Finance	85000.000000
Physics	91000.000000

SELECT *dept_name*, **AVG**(*salary*) **AS** *avg_salary*
FROM *instructor*
WHERE *salary* > 65000
GROUP BY *dept_name*;

dept_name	avg_salary
Biology	72000.000000
Comp. Sci.	83500.000000
Elec. Eng.	80000.000000
Finance	85000.000000
Physics	91000.000000

INSERT

- Add a new tuple to *course*
  - **INSERT INTO** *course*
    VALUES ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- or equivalently
  - **INSERT INTO** *course* (*course_id*, *title*, *dept_name*, *credits*)
    VALUES ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- Add a new tuple to *student* with *tot_creds* set to null
  - **INSERT INTO** *student*
    VALUES ('3003', 'Green', 'Finance', *null*);

UPDATE

- Give a 5% salary raise to all instructors
  - **UPDATE** *instructor*
    SET *salary* = *salary* * 1.05

- Give a 5% salary raise to those instructors who earn less than 70000
  - **UPDATE** *instructor*
    SET *salary* = *salary* * 1.05
    WHERE *salary* < 70000;

- Give a 5% salary raise to instructors whose salary is less than average
  - **UPDATE** *instructor*
    SET *salary* = *salary* * 1.05
    WHERE *salary* < (**SELECT AVG**(salary) **FROM** *instructor*);

DELETE

- Delete all instructors
  - **DELETE FROM** *instructor*;

- Delete all instructors from the Finance department
  - **DELETE FROM** *instructor*
    WHERE *dept_name*= 'Finance';

- Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building
  - **DELETE FROM** *instructor*
    WHERE *dept name* **IN** (**SELECT** *dept name*
    FROM *department*
    WHERE *building* = 'Watson');

Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A subquery is a SELECT-FROM-WHERE expression that is nested within another query
- The nesting can be done in the following SQL query

  SELECT $A_1, A_2, \ldots, A_n$
  FROM $r_1, r_2, \ldots, r_m$
  WHERE P

  as follows:
  - **FROM clause:** r_i can be replaced by any valid subquery
  - **WHERE clause:** P can be replaced with an expression of the form:
    B <operation> (subquery)
    B is an attribute and <operation> is to be defined later
  - **SELECT clause:**
    A_i can be replaced by a subquery that generates a single value (scalar subquery)

Subqueries in the FROM Clause

- Find the average instructors' salaries of those departments where the average salary is greater than $42,000
  - **SELECT** *D.dept_name*, *D.avg_salary*
    FROM (**SELECT** *dept_name*, **AVG**(*salary*) **AS** *avg_salary*
    FROM *instructor*
    GROUP BY *dept_name*) **AS** *D*
    WHERE *D.avg_salary* > 42000;

dept_name	avg_salary
Biology	72000.000000
Comp. Sci.	77333.333333
Elec. Eng.	80000.000000
Finance	85000.000000
History	61000.000000
Physics	91000.000000

Scalar Subquery

- Scalar subquery is used where a single value is expected
  - Runtime error occurs if a subquery returns more than one result tuple
- List all departments along with the number of instructors in each department
  - **SELECT** *dept_name*,
    (**SELECT COUNT**(*)
    FROM *instructor*
    WHERE *department.dept_name* = *instructor.dept_name*)
    AS *num_instructors*
    FROM *department*;

dept_name	num_instructors
Biology	1
Comp. Sci.	3
Elec. Eng.	1
Finance	2
History	2
Music	1
Physics	2

WITH Clause

- The **WITH** clause provides a way of defining a temporary relation
  - The relation is available only to the query in which the **WITH** clause occurs

- Find all departments with the maximum budget
  - **WITH** *max_budget* (*value*) **AS**
    (**SELECT MAX**(*budget*)
    FROM *department*)
    SELECT *department.dept_name*
    FROM *department*, *max_budget*
    WHERE *department.budget* = *max_budget.value*;

dept_name
Finance

Set Membership

- Find courses offered in Fall 2017 and in Spring 2018
  - **SELECT DISTINCT** *course_id*
    FROM *teaches*
    WHERE *semester* = 'Fall' **AND** *year*= 2017 **AND**
    course_id **IN** (**SELECT** *course_id*
    FROM *teaches*
    WHERE *semester* = 'Spring' **AND** *year*= 2018);

course_id
CS-101

Set Membership

- Find courses offered in Fall 2017 but not in Spring 2018
  - **SELECT DISTINCT** *course_id*
    FROM *teaches*
    WHERE *semester* = 'Fall' **AND** *year*= 2017 **AND**
    course_id ***NOT IN*** (**SELECT** *course_id*
    FROM *teaches*
    WHERE *semester* = 'Spring' **AND** *year*= 2018);

course_id
CS-347
PHY-101

Set Membership

- Name all instructors whose name is neither “Mozart” nor Einstein”
  - **SELECT DISTINCT** *name*
    FROM *instructor*
    WHERE *name* **NOT IN** ('Mozart', 'Einstein’);

name
Srinivasan
Wu
El Said
Gold
Katz
Califieri
Singh
Crick
Brandt
Kim

name
Srinivasan
Wu
Mozart
Einstein
El Said
Gold
Katz
Califieri
Singh
Crick
Brandt
Kim

Set Comparison – SOME

- Find names of instructors with salary greater than that of SOME (at least one) instructor in the Biology department
  - **SELECT DISTINCT** *T.name*
    FROM *instructor* **AS** *T*, *instructor* **AS** *S*
    WHERE *T.salary* > *S.salary* **AND** *S.dept name* = 'Biology';

- Same query using > **SOME** clause
  - **SELECT** *name*
    FROM *instructor*
    WHERE *salary* > **SOME** (**SELECT** *salary*
    FROM *instructor*
    WHERE *dept_name* = 'Biology');

name
Wu
Einstein
Gold
Katz
Singh
Brandt
Kim

Set Comparison – ALL

- Find the names of ALL instructors whose salary is greater than the salary of ALL instructors in the Biology department
  - **SELECT** *name*
    FROM *instructor*
    WHERE *salary* > **ALL** (**SELECT** *salary*
    FROM *instructor*
    WHERE *dept name* = 'Biology');

name
Wu
Einstein
Gold
Katz
Singh
Brandt
Kim

Use of NOT EXISTS

- Find all students who have taken all courses offered in the Music department
  - **SELECT DISTINCT** *S.ID*, *S.name*
    FROM *student* **AS** *S*
    WHERE NOT EXISTS (**SELECT** *course_id*
    FROM *course*
    WHERE *dept_name* = 'Music'
    AND *course_id* **NOT IN**
    (**SELECT** *T.course_id*
    FROM *takes* **AS** *T*
    WHERE *S.ID* = *T.ID*));

ID	name
55739	Sanchez

SQL Order of Execution

Order	Clause	Function
1	FROM	Choose and join tables to get base data
2	WHERE	Filters the base data
3	GROUP BY	Aggregates the base data
4	HAVING	Filters the aggregated data
5	SELECT	Returns the final data
6	ORDER BY	Sorts the final data
7	LIMIT	Limits the returned data to a row count

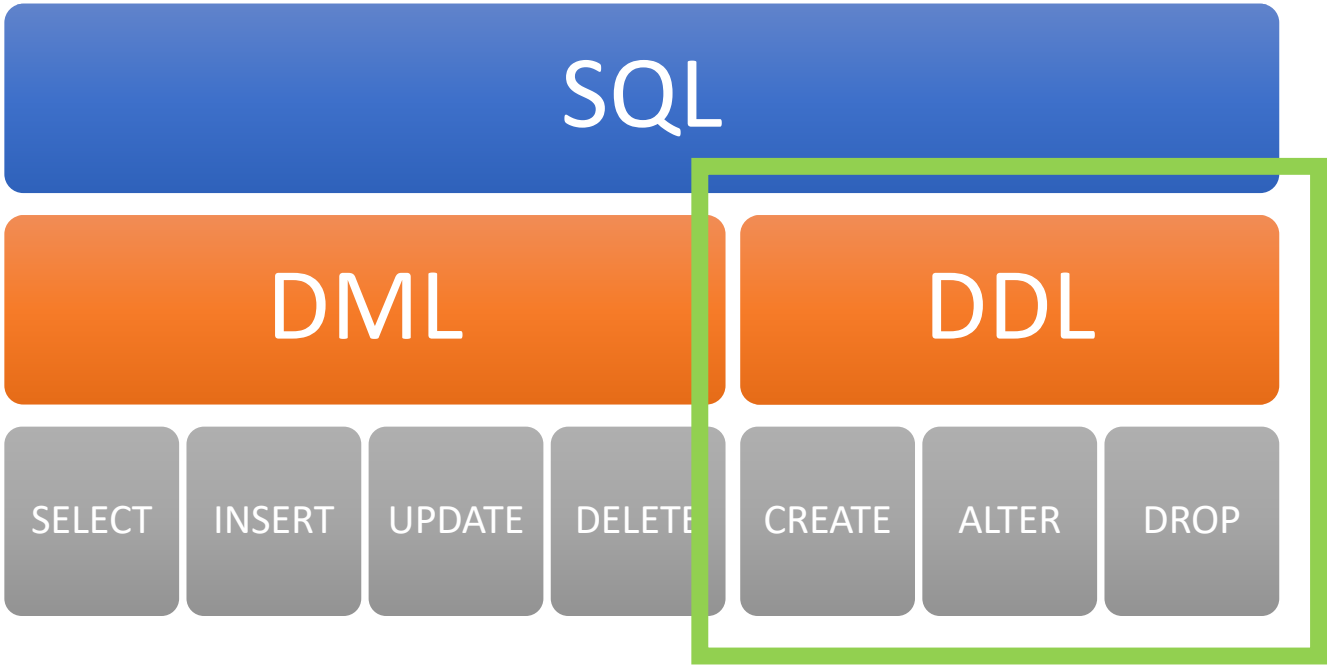
```
SELECT developer, COUNT(*) AS GamesCount, AVG(Price) AS AvgPrice
FROM games
JOIN distributors ON games.dist_id = distributors.dist_id
WHERE release_year > 2019
GROUP BY developer
HAVING COUNT(*) > 2
ORDER BY AvgPrice DESC;
```

SQL Order of Execution

- Explanation
  1. **FROM** games: The data source is defined
  2. **JOIN** distributors **ON** games.dist_id = distributors.dist_id: Tables are joined to include distributor data in the results
  3. **WHERE** release_year > 2019: Rows are filtered based on the release year of the games
  4. **GROUP BY** developer: The results are grouped by the developer
  5. **HAVING COUNT**(*) > 2: The groups are filtered based on the count of games per developer
  6. **SELECT** developer, **COUNT**(*) AS GamesCount, **AVG**(Price) **AS** AvgPrice: Specific columns and calculated columns are selected
  7. **ORDER BY** AvgPrice **DESC**: The final result set is ordered by the average price in descending order

Agenda

- SQL DDL (Data Definition Language)



Agenda

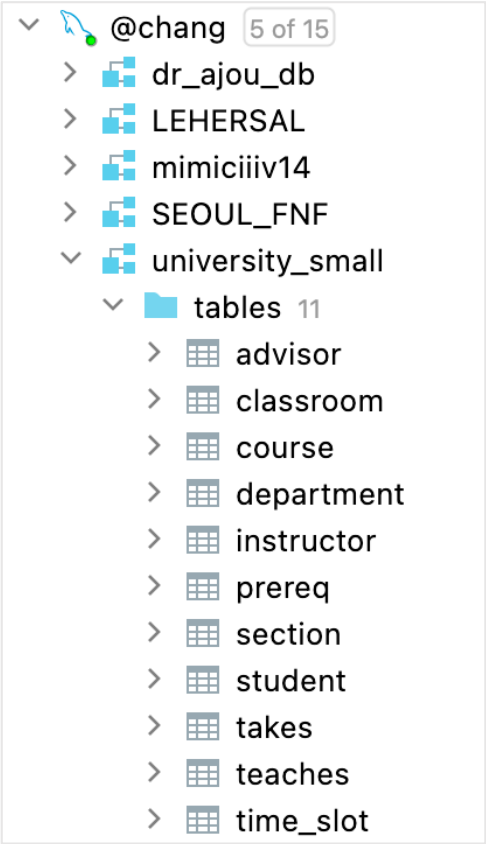
- SQL DDL (Data Definition Language)
  - Commands
  - Data types
    - String data types
    - Numeric data types
    - Temporal data types
    - Large objects
  - Integrity constraints
    - Primary key constraints
    - Foreign key constraints
    - Unique constraints
    - Not null, default, check constraints

Data Definition Language

- The SQL data-definition language (DDL) allows the specification of information about relations, including:
  - The schema for each relation
  - The type of values associated with each attribute
  - The Integrity constraints
  - The set of indices to be maintained for each relation
  - Security and authorization information for each relation
  - The physical storage structure of each relation on disk

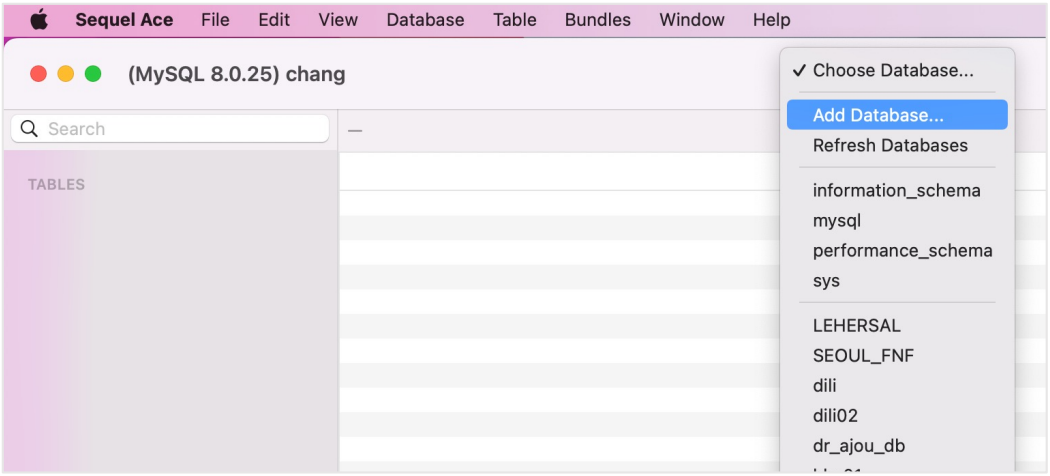
CREATE DATABASE

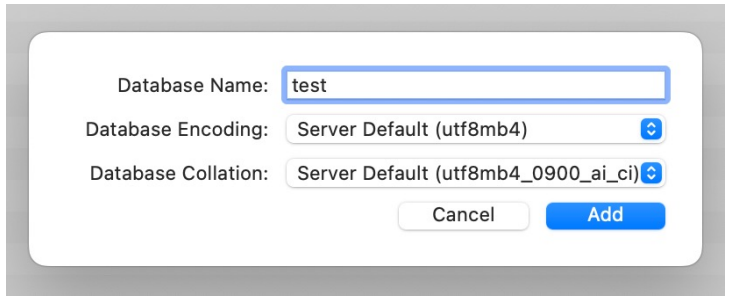
- To initialize a new database
- Basic syntax:
  CREATE DATABASE *database_name*
  - One can specify the default character encoding method along with this command
    - **CREATE DATABASE** test
      DEFAULT CHARACTER SET utf8
      COLLATE utf8_unicode_ci;
      - Collation: a set of rules that defines how to compare and sort character strings
  - After creating a database, to use it
    USE *database_name*
  - Ref: https://dev.mysql.com/doc/refman/8.4/en/charset.html

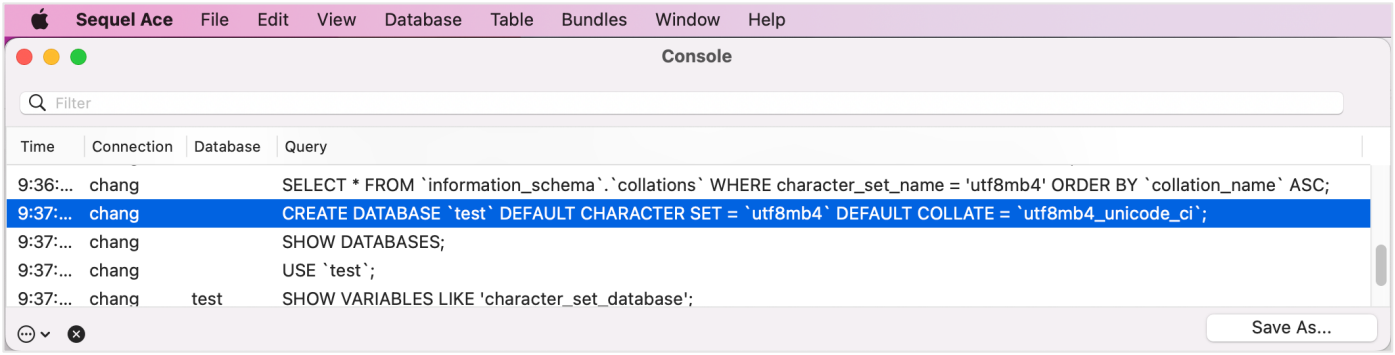


Example: Creating a Database on Sequel Pro

- Creating a new database







View > Show Console

CREATE TABLE

- To create a new table
- Basic syntax:

  CREATE TABLE *table_name*(
  Col1_name data_type[(size)],
  Col2_name data_type[(size)]
)

- E.g., Creating a table with four columns
  - **CREATE TABLE** *books*(
    ISBN CHAR(20),
    Title CHAR(50),
    AuthorID INTEGER,
    Price FLOAT)

Table Updates (Updating Table Schemas)

- DROP TABLE
  - **DROP TABLE** *r*
    - Remove relation *r*

- ALTER
  - **ALTER TABLE** *r* **ADD** *A D*
    - *A* is the name of the new attribute to add to relation *r*; *D* is the domain of *A*
    - All existing tuples in the relation are assigned *null* as the value for the new attribute

  - **ALTER TABLE** r **DROP** *A*
    - *A* is the name of an attribute in *r*
    - Dropping of attributes not supported by some old databases (MySQL does)

C.f., Table Updates (Updating Tuples)

- INSERT
  - **INSERT INTO** *instructor* **VALUES** ('10211', 'Smith', 'Biology', 66000)

- DELETE
  - **DELETE FROM** *student*
    - Remove all tuples from the *student* relation

⇒ *Here we are talking about schemas* (*data definition*)

Table Updates (Updating Table Schemas)

- Examples
  - **DROP TABLE** time_slot_backup;
  - **ALTER TABLE** time_slot_backup **ADD** remark VARCHAR(20);
  - **ALTER TABLE** time_slot_backup **DROP** remark;

Data Definition Language (revisited)

- The SQL data-definition language (DDL) allows the specification of information about relations, including:
  - The schema for each relation
  - The type of values associated with each attribute
  - The Integrity constraints
  - The set of indices to be maintained for each relation
  - Security and authorization information for each relation
  - The physical storage structure of each relation on disk

Agenda

- SQL DDL (Data Definition Language)
  - Commands
  - **Data types**
    - String data types
    - Numeric data types
    - Temporal data types
    - Large objects
  - Integrity constraints
    - Primary key constraints
    - Foreign key constraints
    - Unique constraints
    - Not null, default, check constraints

Domain Types in SQL

- SQL Data Types
  - **CHAR(*n*)**: Fixed length character string, with user-specified length *n*
    - Maximum length *n* = [0, 255]
  - **VARCHAR(*n*)**: Variable length character strings, with user-specified maximum length *n*
    - Maximum length *n* = [0, 65,535]
    - *If the length is always the same, use a CHAR-type attribute;*
      if you are storing wildly variable length strings, use a VARCHAR-type attribute
  - **TEXT**: for strings longer than the range of VARCHAR
    - TINYTEXT 0 – 255 bytes
    - TEXT 0 – 65,535 bytes
    - MEDIUMTEXT 0 – 16,777,215 bytes
    - LONGTEXT 0 – 4,294,967,295 bytes

String Data in SQL

- Difference between CHAR and VARCHAR

Value	CHAR(4)	Storage	VARCHAR(4)	Storage
‘’	‘ ‘	4 bytes	‘’	1 bytes
‘ab’	‘ab ‘	4 bytes	‘ab’	3 bytes
‘abcd’	‘abcd’	4 bytes	‘abcd’	5 bytes
‘abcdefg’	‘abcd’	4 bytes	‘abcd’	5 bytes

Domain Types in SQL

- SQL Data Types
  - **INT, INTEGER**: Integer (a finite subset of the integers that is machine-dependent)
  - **SMALLINT**: Small integer (a machine-dependent subset of the integer domain type)
  - **BIGINT**: Big integer (a machine-dependent subset of the integer domain type)

  - **TINYINT** and **MEDIUMINT** are also available

Domain Types in SQL

- Different R-DBMSs support different combinations of those integer types

	Bytes	MySQL	MS SQL	PostgresSQL	DB2
TINYINT	1	✓	✓		
SMALLINT	2	✓	✓	✓	✓
MEDIUMINT	3	✓			
INT/INTEGER	4	✓	✓	✓	✓
BIGINT	8	✓	✓	✓	✓

  - *C.f.*, Oracle only has a NUMBER datatype

Domain Types in SQL

- SQL Data Types
  - **NUMERIC(*p*,*d*)**: Fixed point number (exact value) with user-specified precision of *p* digits, with *d* digits to the right of decimal point
    - *E.g.*, **NUMERIC**(3,1) allows 44.5 to be stores exactly, but not 444.5 or 0.32)
    - In MySQL, **DECIMAL** is NUMERIC
  - **FLOAT**: Floating point number (approximate) with single-precision
  - **REAL, DOUBLE**: Floating point number (approximate) with double-precision

Domain Types in SQL

- DECIMAL vs INT/FLOAT/DOUBLE
  - FLOAT and DOUBLE are faster than DECIMAL
  - DECIMAL values are exact
    - Example

floats: FLOAT	decimals: DECIMAL(3,2)
1.1	1.10
1.1	1.10
1.1	1.10

- SELECT SUM(…) → DECIMAL values are precise

SUM(floats)	SUM(decimals)
3.3000000715255737	3.30

Domain Types in SQL

- SQL Data Types
  - **DATE**: ‘YYYY-MM-DD’
    - Rage: 1000-01-01 to 9999-12-31
    - *E.g.*, ‘2020-03-01’ for March 1, 2020
  - **TIME**: ‘HH:MM:SS’
    - Range: -838:59:59 to 838:59:59
    - *E.g.*, ’14:30:03.5’ for 3.5 seconds after 2:30pm
  - **DATETIME**: ‘YYYY-MM-DD HH:MM:SS’
    - Range: 1000-01-01 00:00:00 to 9999-12-31 23:59:59
  - **YEAR**: ‘YYYY’
    - Range: 1901 to 2155, or 0000 (illegal year values are converted to 0000)

Temporal Data in SQL

- SQL Data Types
  - **TIMESTAMP(*n*)**: Unix time (time since Jan 1, 1970)
    - A way to track time as a running total of seconds
    - Range: 1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC
    - Typically used for logging (keeping records of all the system events)
    - *URL: https://time.is/Unix*

UNIX
TIME
SINCE 1970

Temporal Data in SQL

- SQL Data Types
  - **TIMESTAMP(*n*)**: Unix time (time since Jan 1, 1970)
    - A way to track time as a running total of seconds
    - Range: 1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC
    - Typically used for logging (keeping records of all the system events)
    - *URL: https://time.is/Unix*

Binary : 01111111 11111111 11111111 11110000

Decimal : 2147483632

Date : 2038-01-19 03:13:52 (UTC)

Date : 2038-01-19 03:13:52 (UTC)

On Linux

On DBMS'

Domain Types in SQL

- SQL Data Types
  - **TIMESTAMP(*n*)**: Unix time (time since Jan 1, 1970)
    - Range: 1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC
    - Typically used for logging (keeping records of all the system events)

    - Depending on size *n*, the display pattern changes

	Format
TIMESTAMP(14)	YYYYMMDDHHMMSS
TIMESTAMP(12)	YYMMDDHHMMSS
TIMESTAMP(10)	YYMMDDHHMM
TIMESTAMP(8)	YYYYMMDD
TIMESTAMP(6)	YYMMDD
TIMESTAMP(4)	YYMM
TIMESTAMP(2)	YY

Domain Types in SQL

- SQL Data Types
  - **BINARY(*n*)**: binary byte data type, with user-specified length *n*
    - Contains a byte strings (rather than a character string)
    - Maximum length *n* = [0, 255]
  - **VARBINARY(*n*)**: binary byte data type, with user-specified maximum length *n*
    - Maximum length *n* = [0, 65,535]

  - **BLOB**: Binary Large OBject data type
    - TINYBLOB 0 – 255 bytes
    - BLOB 0 – 65,535 bytes (65 KB)
    - MEDIUMBLOB 0 – 16,777,215 bytes (16 MB)
    - LONGBLOB 0 – 4,294,967,295 bytes (4 GB)

Agenda

- SQL DDL (Data Definition Language)
  - Commands
  - Data types
    - String data types
    - Numeric data types
    - Temporal data types
    - Large objects
  - **Integrity constraints**
    - Primary key constraints
    - Foreign key constraints
    - Unique constraints
    - Not null, default, check constraints

CREATE TABLE Construct

- A new relation is defined using the **CREATE TABLE** command:

  CREATE TABLE *r*

  (A_1 D_1, A_2 D_2, …, A_n D_n,
  (*integrity-constraint*$_1$),
  …,
  (*integrity-constraint*$_k$))

  - *r* is the name of the relation
  - Each A_i is an attribute name in the schema of relation *r*
  - Each D_i is the data type of values in the domain of attribute A_i

- Example:

```
CREATE TABLE instructor(
        ID          CHAR(5),
        name        VARCHAR(20),
        dept_name   VARCHAR(20),
        salary      NUMERIC(8,2))
```

Integrity Constraints in CREATE TABLE

- SQL prevents any update to the database that violates an integrity constraint
  - Integrity constraints allow us to specify what data makes sense for us
- Types of integrity constraints
  - Primary key: **PRIMARY KEY** (A_1, ..., A_n)
  - Foreign key: **FOREIGN KEY** (A_m, ..., A_n) **REFERENCES** *r*
  - Unique key: **UNIQUE**
  - Not null: **NOT NULL**

- Example:

```
CREATE TABLE instructor(
        ID              CHAR(5),
        name            VARCHAR(20) NOT NULL,
        dept_name       VARCHAR(20)
        salary          NUMERIC(8, 2),
        PRIMARY KEY (ID),
        FOREIGN KEY (dept_name) REFERENCES department);
```

Declaring Keys

- An attribute or list of attributes may be declared as PRIMARY KEY or UNIQUE
  - Meaning: no two tuples of the relation may agree in all the attribute(s) on the list
    - That is, the attribute(s) do(es) not allow duplicates in values
    - PRIMARY KEY/UNIQUE can be used as an identifier for each row
  - Comparison: PRIMARY KEY vs UNIQUE

PRIMARY KEY	UNIQUE
Used to serve as a unique identifier for each row in a relation	Uniquely determines a row which is not primary key
Cannot accept NULL	Can accept NULL values (some DBMSs accept only one NULL value)
A relation can have only one primary key	A relation can have more than one unique attributes
Clustered index	Non-clustered index

Integrity Constraints

- **NOT NULL** – disallowing null values
  - Null values indicate that the data is not known
  - These can cause problems in querying database
  - The Primary Key columns automatically prevent null being entered
  - *C.f.*, **NULL** – can be used to explicitly allow null values

```
CREATE TABLE studio (
    ID          NUMERIC(5,0) PRIMARY KEY,
    name        VARCHAR(20) NOT NULL,
    city        VARCHAR(20) NULL,
    state       CHAR(2) NOT NULL
);
```

Integrity Constraints

- **DEFAULT** – A default value can be inserted in any column with this keyword
  - *E.g.,* **CREATE TABLE** *movies*(

```
        movie_title     VARCHAR(40) NOT NULL,
        release_date    DATE  DEFAULT sysdate  NULL,
        genre           VARCHAR(20) DEFAULT 'Comedy'
                        CHECK genre IN ('Comedy', 'Action', 'Drama')
    )
```

  - In MySQL,
    - **CREATE TABLE** *movies*(

```
        movie_title     VARCHAR(40) NOT NULL,
        release_date    DATE  DEFAULT CURRENT_TIMESTAMP  NULL,
        genre           VARCHAR(20) DEFAULT 'Comedy'
                        CHECK genre IN ('Comedy', 'Action', 'Drama')
    )
```

Integrity Constraints

- **CHECK** – Allows the inserted value to be checked
  - *E.g.*, **CREATE TABLE** *movies*(

```
        movie_title      VARCHAR(40) PRIMARY KEY,
        release_date     DATE,
        budget           INTEGER CHECK (budget > 50000)
    )
```

  - Table-level constraints can be defined; *E.g.*,
    - **CREATE TABLE** *movies*(

```
        movie_title      VARCHAR(40) PRIMARY KEY,
        release_date     DATE,
        budget           INTEGER CHECK (budget > 50000),
        CONSTRAINT release_date_const
        CHECK (release_date BETWEEN '01-Jan-2000' AND '31-Dec-2009')
    )
```

Declaring Keys

- **CREATE TABLE** *student* (
  ID **VARCHAR**(5),
  name **VARCHAR**(20) **NOT NULL**,
  dept_name **VARCHAR**(20),
  tot_cred **NUMERIC**(3,0),
  PRIMARY KEY *(ID),*
  FOREIGN KEY *(dept_name)* **REFERENCES** *department*);

- **CREATE TABLE** *student* (
  ID **VARCHAR**(5) **PRIMARY KEY**,
  name **VARCHAR**(20) **NOT NULL**,
  dept_name **VARCHAR**(20),
  tot_cred **NUMERIC**(3,0),
  FOREIGN KEY *(dept_name)* **REFERENCES** *department*);

More Examples

- **CREATE TABLE** *takes* (

```
        ID              VARCHAR(5),
        course_id       VARCHAR(8),
        sec_id          VARCHAR(8),
        semester        VARCHAR(6),
        year            NUMERIC(4,0),
        grade           VARCHAR(2),
        PRIMARY KEY (ID, course_id, sec_id, semester, year),
        FOREIGN KEY (ID) REFERENCES  student,
        FOREIGN KEY (course_id, sec_id, semester, year)
                REFERENCES section);
```

More Examples

- **CREATE TABLE** *course* (

  course_id **VARCHAR**(8),
  title **VARCHAR**(50),
  dept_name **VARCHAR**(20) **DEFAULT** ‘Comp. Sci’,
  credits **NUMERIC**(2,0),
  PRIMARY KEY *(course_id),*
  FOREIGN KEY *(dept_name*) **REFERENCES** *department*);

More Examples

- **CREATE TABLE** *neighbors*(
  name **CHAR**(30) **PRIMARY KEY**,
  addr **CHAR**(50) **DEFAULT** '123 Sesame St.',
  phone **CHAR**(16));

  - Inserting Elmo is a neighbor:
    - INSERT INTO neighbors (name)
      VALUES ('Elmo');

name	addr	phone
'Elmo'	'123 Sesame St.'	NULL

More Examples

- **CREATE TABLE** *neighbors*(
  name **CHAR**(30) **PRIMARY KEY**,
  addr **CHAR**(50) **DEFAULT** '123 Sesame St.',
  phone **CHAR**(16) **NOT NULL**);

  - Inserting Elmo is a neighbor:
    - INSERT INTO neighbors (name)
      VALUES ('Elmo');

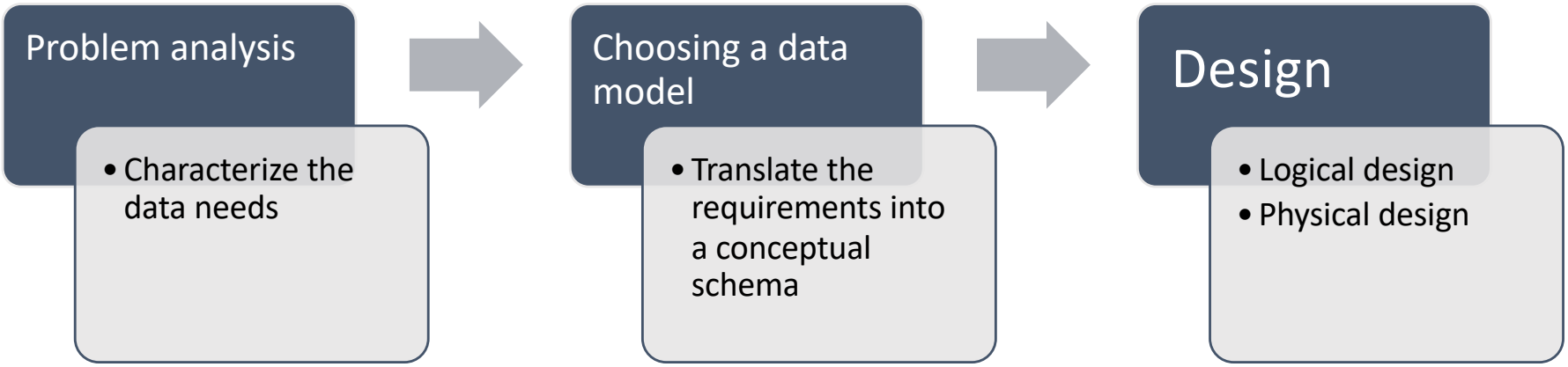
      ➔ If phone were NOT NULL, this insertion would have been rejected

Agenda

- Designing a database
- E-R diagrams

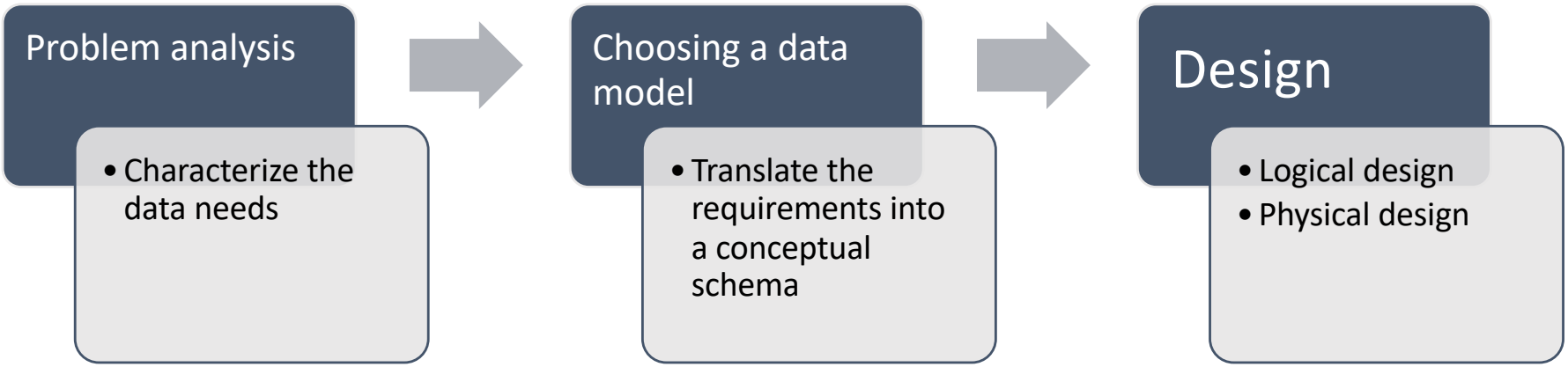
Design Phases

- Initial phase: characterize fully the data needs of the prospective database users
- Second phase: choose a data model
  - Apply the concepts of the chosen data model
  - Translate the requirements into a conceptual schema of the database
  - A fully developed conceptual schema indicates the functional requirements of the enterprise
    - Describe the kinds of operations (or transactions) that will be performed on the data



Design Phases

- Final Phase: Move from an abstract data model to the implementation of the database
  - Logical Design – Deciding on the database schema
    - Database design requires that we find a "good" collection of relation schemas
    - *Business decision – What attributes should we record in the database?*
    - *Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?*
  - Physical Design – Deciding on the physical layout of the database



Design Phases

- In designing a database schema, we must ensure that we avoid two major pitfalls:
  - Redundancy: a bad design may result in repeated information
    - Redundant representation of information may lead to data inconsistency among the various copies of information
  - Incompleteness: a bad design may make certain aspects of the enterprise difficult or impossible to model

- Avoiding bad designs is not enough. There may be a large number of good designs from which we must choose

Design Approaches

- Entity Relationship Model
  - Models an enterprise as a collection of *entities* and *relationships*
    - Entity: a "thing" or "object" in the enterprise that is distinguishable from other objects
      - Described by a set of *attributes*
    - Relationship: an association among several entities
  - Represented diagrammatically by an *entity-relationship diagram* (E-R diagram)

- Normalization Theory
  - Formalize what designs are bad, and test for them